A. Mảng đánh dấu

(Marker array)

Tổng quát: Mảng đánh dấu là một cấu trúc dữ liệu quan trọng trong Lập trình thi đấu, thường được dùng để theo dõi trạng thái hoặc thông tin về một tập hợp các phần tử có trật tự nhất định. Hãy cùng tìm hiểu về các kỹ thuật đánh dấu trong bài học này nhé!

I. Khái niệm:

- Mảng đánh dấu là mảng một (hoặc nhiều) chiều. Trong đó, mỗi phần tử được sử dụng để đại diện cho một giá trị hoặc trạng thái, thường là giá trị nhị phân (***true*** hoặc ***false***).
- Tác dụng: Theo dõi trạng thái xuất hiện/vắng mặt của các phần tử trong một tập hợp.
- Ví dụ: Để đánh dấu một giá trị ***x***, ta cập nhật trạng thái cho *phần tử ở vị trí **x*** trong mảng đánh dấu.

II. Đặc điểm điểm:

- **Tiết kiệm bộ nhớ:** Mảng đánh dấu tiêu tốn ít bộ nhớ hơn so với một mảng chứa các đối tượng lớn.
- **Xử lý nhanh:** Kiểm tra và cập nhật trạng thái của một phần tử trong mảng một cách dễ dàng và nhanh chóng.
- **Thông dụng:** Thường dùng trong các bài toán liên quan đến kiểm tra sự xuất hiện, thống kê, hoặc xác định tính chất, điều kiện nào đó của các phần tử.
- **Nhược điểm:** Vì dùng giá trị làm chỉ số, mảng đánh dấu chỉ thao tác được với các giá trị nguyên dương bé hơn hoặc bằng 10^8.

III. Ứng dụng:

1. Sử dụng:

Đánh dấu phần tử	Kiểm tra trạng thái

`bool marker[1001];`
`int x; cin >> x;`
`marker[x] = true;`
	`if (marker[x]) cout << x << " is marked";`
	`else cout << x << " is not marked";`

Vận dụng: Nhập vào một dãy có ***n*** số nguyên *(n ⩽ 1000)*. Kiểm tra trong dãy đã nhập vào có những phần tử nào thuộc *[1; 1000]*.	

```
#include <bits/stdc++.h>
using namspace std;
const int MAX_VAl = 1000;
bool marker[MAX_VAL + 1] = {false}; // gán tất cả phần tử = false
int main() {
    // NHẬP và ĐÁNH DẤU:
    int n; cin >> n;
    for (int i=1; i<=n; i++) {
        int x; cin >> x;
        marker[x] = true;
    }
    // In các phần tử ĐÃ ĐƯỢC ĐÁNH DẤU:
    for (int i=1; i <= MAX_VAL; i++)
        if (marker[i]) cout << i << " is marked";
    return 0;
}
```

2. Sàng nguyên tố Eratosthenes:

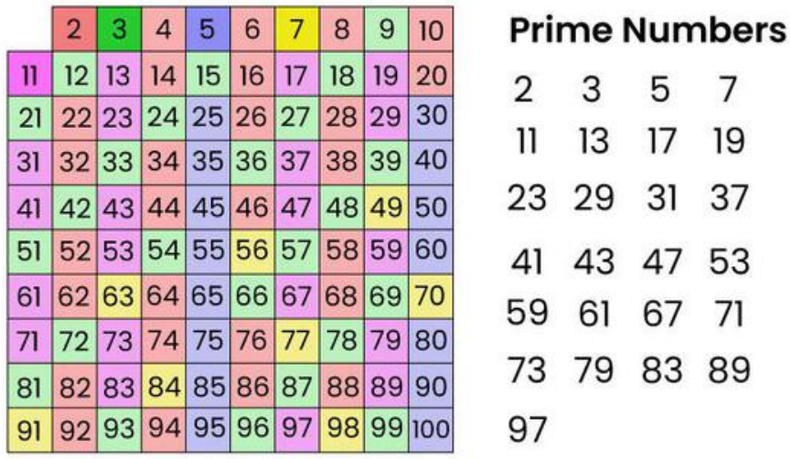
Giới thiệu: Eratosthenes (276 TCN - 194 TCN) là nhà toán học, địa lý và thiên văn người Hy Lạp. Những người đương thời gọi ông là "Beta" do ông là người nổi tiếng thứ hai trên thế giới vào thời điểm đó trong nhiều lĩnh vực, đứng sau nhà vật lý đại tài Archimedes. *(Xem thêm về Eratosthenes)*

Sàng Eratosthenes: Ban đầu, Eratosthenes đã ghi ra các số từ 2 đến 100. Sau đó, ông đánh dấu các *hợp số* và giữ nguyên các *số nguyên tố*. Sau cùng, toàn bộ các số chưa được đánh dấu đều là số nguyên tố.

Thực hiện:

- **B1:** Tạo danh sách các số tự nhiên liên tiếp từ ***2*** đến ***n***: *(2, 3, 4, ..., n)*.
- **B2:** Giả sử tất cả số trong danh sách đều là số nguyên tố *(đặt tất cả = **TRUE**)*. Trong đó, ***p*** = ***2*** là số nguyên tố đầu tiên.
- **B3:** Tìm và đánh dấu tất cả các bội số của ***p*** *(2p, 3p, 4p,...)*, những số bị đánh dấu gán = ***FALSE***.
- **B4:** Tìm *số nhỏ nhất lớn hơn **p** chưa bị đánh dấu và bé hơn hoặc bằng **sqrt(n)*** (Lúc này, tất cả các số chưa bị đánh dấu đảm bảo không có bội của ***p***). Ta gọi số vừa tìm được là số ***p*** mới và lặp lại B3.

→ Đến khi số ***p*** đạt đến ***sqrt(n)*** thì dừng tìm kiếm. Khi đó, ta có được danh sách với toàn bộ các số chưa bị đánh dấu là số nguyên tố.



→ Để dễ hình dung, vui lòng xem GIF [tại đây](#).

- **Code mẫu: Độ phức tạp O(n * log(log(n)))**

```
void sieve(int N) {
    bool prime[N+1] = {true};
    prime[0] = prime[1] = false;
    for (int i=2; i*i <= N; ++i)
        if (prime[i])
            for (int j = i*i; j<=N; j+=i)
                prime[j] = false; // đánh dấu các bội của i = false
}
```

→ Kỹ thuật sàng Eratosthenes sẽ giúp bạn xử lý được những bài toán có số lớn một cách tối ưu.

Lưu ý: Bài viết chỉ nói về ý tưởng vận dụng mảng đánh dấu trong sàng Eratosthenes. Từ ý tưởng này, các bạn có thể tạo được các sàng khác: sàng ước, sàng chính phương,... để giải quyết những bài toán có yếu tố tương tự.

>>> Nguồn tham khảo (phần A):

- *Wikipedia: vi.wikipedia.org/wiki/S%C3%A0ng_Eratosthenes*
- *Geeks for Geeks: geeksforgeeks.org/sieve-of-eratosthenes*
- *VNOI: vnoi.info/wiki/translate/he/Number-Theory-2.md*

Mảng đánh dấu tuy không khó học nhưng có rất nhiều dạng bài tập và nhiều cách vận dụng khác nhau, do đó mà bài viết này khác dài. Bạn có thể chia ra đọc trong nhiều ngày để học hiệu quả hơn.
Hãy đảm bảo rằng bạn đã nắm tương đối kĩ phần A và đã sẵn sàng đến với phần tiếp theo.

B. Mảng đếm

(Counter array)

I. Khái niệm:

- Mảng đếm là một mảng một (hoặc nhiều) chiều trong đó mỗi phần tử là một biến đếm, thường được sử dụng để đếm số lần xuất hiện của các giá trị cụ thể.
- Sử dụng giá trị làm chỉ số: Các giá trị trong dãy dữ liệu thường được sử dụng làm chỉ số của mảng đếm (tương tự mảng đánh dấu).

II. Đặc điểm:

- Ưu điểm:

- Dễ hiểu và triển khai. Hiệu suất tốt trong việc đếm và theo dõi tần suất xuất hiện của các giá trị.
- Thường được sử dụng để thống kê dữ liệu, số lần xuất hiện của các phần tử.

- Nhược điểm: Vì dùng giá trị làm chỉ số, mảng đếm chỉ thao tác được với các giá trị nguyên dương bé hơn hoặc bằng 10^8.

III. Ứng dụng:

Xây dựng	Kiểm tra trạng thái

`int counter[1001];`
`int x; cin >> x;`
`counter[x]++;`
	`cout << x << " appeared" << counter[x] << " time(s)";`

Vận dụng: Nhập vào một dãy có ***n*** số nguyên *(n ⩽ 1000)*. Với mỗi phần tử, in ra số lần xuất hiện của nó trong dãy.

```
#include <bits/stdc++.h>
using namspace std;
const int MAX_VAl = 1000;
bool counter[MAX_VAL + 1] = {0}; // gán tất cả phần tử = 0
int main() {
    // NHẬP và ĐẾM:
    int n; cin >> n;
    for (int i=1; i<=n; i++) {
        int x; cin >> x;
        counter[x]++;
    }
    // In SỐ LẦN XUẤT HIỆN của mỗi phần tử (có trùng lặp):
    for (int i=1; i <= n; i++)
        cout << x << " appeared" << counter[x] << " time(s)\n";
    return 0;
}
```

Thật sự không khó để hiểu được mảng đếm. Đây là một trong những kỹ thuật thống kê rất cơ bản và phổ biến. Hẳn là bạn đã từng sử dụng qua trước khi đọc bài này. Tuy nhiên, để khắc phục những hạn chế nêu trên, một kiểu dữ liệu thần thánh trong thư viện chuẩn (STL) của C++ đã ra đời...

C. std::map trong C++

Map trong C++ là một tập hợp các phần tử được sắp xếp theo thứ tự cụ thể, mà mỗi phần tử trong đó được hình thành bởi sự kết hợp của một cặp khóa và giá trị (key & value), với mỗi khóa là duy nhất trong map.

- Ví dụ 1: Cho dãy số *0, 2, -3, 9, 1, 2, 1, 0, 0, 7* có ***n*** phần tử. Với map, nếu đưa các phần tử trong dãy trên vào key và số lần xuất hiện của chúng là value, ta được:

	0	1	2	3	4	5
key	-3	0	1	2	7	9
value	1	3	2	2	1	1

Phân tích: (vừa quan sát hình ảnh trên vừa đọc)

Cài đặt	Nhận xét

`map <int, int> mp;`
`for (int i=1; i<=n; i++) {`
`  int x; cin >> x;`
`  mp[x]++; // như mảng đếm`
`}`
`for (auto k : mp)`
`  cout << k.first << " " << k.second << '\n';`
`return 0;`
	• Map tự động sắp xếp khóa key từ bé đến lớn mà không cần cài đặt hàm sort.
	• Mỗi giá trị key chỉ được lưu 01 lần (không có trùng lặp).

- Ví dụ 2: Cho danh sách tên học sinh lớp 9A có ***n*** học sinh và số điểm 10 mà mỗi em đạt được trong học kỳ 1. Hãy tìm học sinh có nhiều điểm 10 nhất và xác định số điểm 10 của bạn ấy.

INPUT	OUTPUT
4 An 7 Nam 4 Minh 6 Vy 9	Vy 9

Bài giải:

<table>
<tr><th>Cài đặt</th><th>Nhận xét</th></tr>
<tr><td>

```
map <string, int> mp;
for (int i=1; i<=n; i++) {
    string s; cin >> s;
    mp[s]++;
}
string name; int val = 0;
for (auto k : mp) {
    if (k.second > val) {
        val = k.second; // truy cập key
        name = k.first; // truy cập value
    }
}
cout << name << " " << val;
return 0;
```

</td><td>

mp	key	value
0	An	7
1	Minh	6
2	Nam	4
3	Vy	9

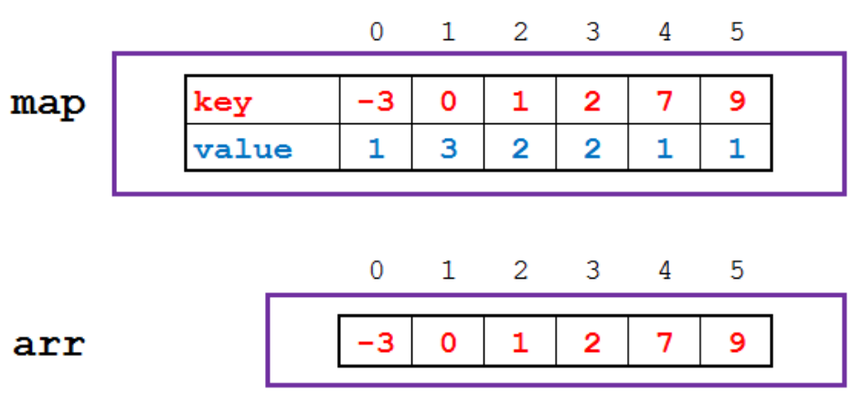
- Map tự động sắp xếp khóa key theo thứ tự tăng dần của bảng chữ cái mà không cần cài đặt hàm sort.
- Mỗi giá trị key chỉ được lưu 01 lần.

</td></tr>
</table>

- Lưu ý:

- Khi cần map không tự động sort khóa key, bạn có thể dùng `unordered_map`. Các cấu trúc và cách sử dụng của `unordered_map` tương tự `map`.
- `map` và `unordered_map` có thể lưu mọi kiểu dữ liệu cơ bản.
- Mỗi giá trị key chỉ được lưu 1 lần (không có trùng lặp).
- Ngoài tác dụng cơ bản là đếm, `map` có thể dùng kết hợp với các kỹ thuật lập trình khác như: đánh dấu, cộng dồn,hãy tìm kiếm nhị phân,...
- Không nên phụ thuộc vào `map` bởi vì nó khiến thời gian chạy chương trình chậm đi. Bạn chỉ nên dùng `map` khi cần thao tác với các KDL đặc biệt, còn bình thường hãy ưu tiên dùng mảng đánh dấu, mảng đếm.

- So sánh map và mảng thông thường:



- Một `map` bao gồm cả hai khóa `key` và `value`, tức là khi truy cập vào một *iterator* của `map`, bạn có thể thao tác với một trong hai hoặc cả hai khóa. Đặc biệt, `map` không có kích thước cụ thể, giới hạn của `map` phụ thuộc vào bộ nhớ hệ thống.
- Mảng là một dãy giá trị cùng kiểu dữ liệu và có độ dài nhất định.

- Ưu điểm:

- **Tìm kiếm hiệu quả:** Map cung cấp việc tìm kiếm theo khóa với độ phức tạp thời gian tốt, làm cho mảng đánh dấu nhanh chóng và hiệu quả.

- **Dễ sử dụng:** Map giúp quản lý dễ dàng và có thể mở rộng mảng đánh dấu một cách linh hoạt.
- **Tiện lợi:** Map thao tác được với nhiều kiểu dữ liệu (đặc biệt là số âm) giúp bạn giải quyết được các bài toán dùng kỹ thuật đánh dấu mà không cần thêm bước xử lý dữ liệu đặc biệt.

>>> Nguồn tham khảo (phần B và C):

- *Lập trình căn bản: laptrinhcanban.com/cpp/lap-trinh-cpp-co-ban/map-trong-cpp*
- *VNCoder: vncoder.vn/bai-hoc/map-khai-niem-co-ban-291*
- *Code Learn: codelearn.io/sharing/cau-truc-du-lieu-than-thanh-mang-ten-map*
- *Viblo: viblo.asia/p/set-va-map-trong-c-djeZ1jPJlWz*

D. Bài tập vận dụng

Bài tập 1: Bác kỹ sư chăm chỉ

Một nhà địa lý học đang xem xét số liệu về số liệu chấn động của mặt đất qua từng ngày. Số liệu có dạng một mảng ***A***. Để tiện cho việc nghiên cứu ông ấy cần tính xác suất xuất hiện của tần số ***X*** trong ***N*** ngày. Với vai trò là trợ lý bạn phải trả lời *Q* câu hỏi cho giáo sư để ông ấy hoàn thành công việc.

Dữ liệu:

- Dòng đầu tiên là ***N*** và ***Q*** $(0 < N, Q \le 10^6)$ là độ dài của mảng ***A*** và số câu hỏi.
- Dòng thứ hai gồm ***N*** số ***A[i]*** $(0 \le |A[i]| \le 10^6)$ là các giá trị của mảng ***A***.
- Q dòng kế tiếp là con số ***X*** $(0 \le |X| \le 10^6)$.

Kết quả: Gồm Q dòng mỗi dòng là phần trăm xuất hiện của số ***X*** tương ứng làm tròn 2 chữ số.

INPUT	OUTPUT
7 4 1 1 2 2 2 9 -1 1 2 -1 100	28.57 42.86 14.29 0.00

Bài tập 2: spoj.com/problems/PRIME1

Bài tập 3: vnoi.info/problem/nkabd

Bài tập 4: https://short.com.vn/4vdA

E. Tổng kết

✏️ Tài liệu lần này khá dài; ở các phần A, B và C, khuyến khích bạn đọc nên chia phần để đọc và hiểu chứ không nên học hết một lần. Đặc biệt, kiểu dữ liệu map là một trong những sức mạnh lớn nhất của C++ trong bộ môn Lập trình thi đấu, các bạn cần nắm kỹ và luyện tập những dạng bài dùng CTDL này.

✏️ Mảng đánh dấu có thể được sử dụng để giải quyết rất nhiều bài toán ở nhiều mức độ khác nhau. Ở bài học này chỉ giới thiệu về hướng tư duy của thuật toán. Các bạn có thể nghiên cứu thêm ở các đường link nguồn phía trên.

✏️ Về sau, mảng đánh dấu sẽ có đa dạng cách cài đặt và sử dụng hơn. Đặc biệt, giải thuật này có thể được sử dụng xuyên suốt các contest của GG9.

✏️ Đón chờ contest luyện tập sắp tới tại **GDOJ**.

✏️ Để theo dõi dự án, vui lòng tham gia nhóm **Guidelines for Grade 9**.

✏️ Tham gia đoạn chat cộng đồng: m.me/ch/Aba9cpzNk_Td6_fl/ để giao lưu cùng các bạn đồng môn và các anh chị học sinh chuyên Tin trên Tp.HCM.

✏️ Thông tin liên hệ:
- Page: **Căng TIN 2326**
- Group: **Guidelines for Grade 9**
- Gmail: cangtin2326@gmail.com

Chân thành cảm ơn các bạn đã đọc bài viết! ❤️

Mong các bạn tiếp tục ủng hộ GG9 trong thời gian sắp tới 🔥